Exam production webapp

Shared by Matt

This is a copy of a chat between Claude and Matt. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

Report

Can you generate text in a format that will copy-paste into Word as Heading 1 and Heading 2?

Unfortunately, plain text copied into Word won't carry heading styles — Word's Heading 1, Heading 2, etc. are paragraph styles stored in the document itself, not something that can be encoded in plain text or markdown.

Here are your best options:

If you want to paste and have headings automatically applied: Word can recognize heading styles if you paste from a source that carries formatting, like another Word document or a web page with proper HTML heading tags. I can create an actual **.docx file** for you with real Heading 1 and Heading 2 styles applied — you'd just download it and open it directly in Word.

If you're okay with a quick manual step: I can format text using a clear visual convention (like ALL CAPS for H1, title case with a dashed underline for H2), and you can use Word's "Apply Styles" pane to quickly restyle them after pasting.

If you paste from HTML: If you paste content from a browser into Word (using "Keep Source Formatting"), Word will often convert `<h1>` and `<h2>` tags into Heading 1 and Heading 2 styles. I can generate HTML for you to open in a browser and then copy from there.

Would you like me to create a **.docx file** directly, or would the **HTML-to-browser approach** work better for you?

Can you generate a pair of multiple-choice questions in the following format? A paragraph break must separate each question from others. For each question, the question, answer, and distractors for each question must be in the same paragraph.
[Question.] What X does Y? [Answer.] A. [Distractor.] B. [Distractor.] C. [Distractor.] D.
[Question.] What X does Y? [Answer.] A. [Distractor.] B. [Distractor.] C. [Distractor.] D.
The intent is that I will copy-paste into Word for formatting into styles using a later macro.

Here are two sample questions in your format:

What is the primary function of the mitochondria in a cell? Energy production. A. Protein synthesis. B. DNA replication. C. Cell division. D.

What gas do plants absorb during photosynthesis? Carbon dioxide. A. Oxygen. B. Nitrogen. C. Hydrogen. D.

Keep the square-bracket [Question.] [Answer.] [Distractor.] signs, as the later macro will use them.

Here are two sample questions in your format:

[Question.] What is the primary function of the mitochondria in a cell? [Answer.] Energy production. A. [Distractor.] Protein synthesis. B. [Distractor.] DNA replication. C. [Distractor.] Cell division. D.

[Question.] What gas do plants absorb during photosynthesis? [Answer.] Carbon dioxide. A. [Distractor.] Oxygen. B. [Distractor.] Nitrogen. C. [Distractor.] Hydrogen. D.

Remove the letters, so format like this:
[Question.] What is the primary function of the mitochondria in a cell? [Answer.] Energy production. [Distractor.] Protein synthesis. [Distractor.] DNA replication. [Distractor.] Cell division.

[Question.] What is the primary function of the mitochondria in a cell? [Answer.] Energy production. [Distractor.] Protein synthesis. [Distractor.] DNA replication. [Distractor.] Cell division.

[Question.] What gas do plants absorb during photosynthesis? [Answer.] Carbon dioxide. [Distractor.] Oxygen. [Distractor.] Nitrogen. [Distractor.] Hydrogen.

Can you write an HTML page called MCQer that does the following, all on the client side?
(1) The user uploads a docx document where each paragraph is a question formatted like this:
[Question.] What is the primary function of the mitochondria in a cell? [Answer.] Energy production. [Distractor.] Protein synthesis. [Distractor.] DNA replication. [Distractor.] Cell division.
(2) MCQer generats a docx for the user to download. The docx has the following properties:
For each question, the order of the answer and distractors is scrambled.
Each question, answer, and distractor is one its own line, separated by only a paragraph break and no extra breaks.
Each question is formatting with the style Heading 1.
Each answer and distractor is formatted with the style Heading 2.

Show less

Reading docx skill >

Now I have what I need. Since this is a client-side HTML file, I'll use `mammoth.js` to read the uploaded docx and `docx.js` (via CDN) to generate the output docx — all in the browser.

Created a file, read a file >

Here's MCQer. Open the file in any browser — everything runs locally, no server needed.

How it works:

- Upload a `.docx` where each paragraph follows your `[Question.] … [Answer.] … [Distractor.] …` format
- It shows a preview of the questions it finds
- Click **Generate & Download** to get a new `.docx` with the answer and distractors scrambled, questions as Heading 1, and options as Heading 2

The output uses Word's native `Heading 1` / `Heading 2` styles, so your macro will be able to target them normally.

I get the following: Error generating document: docx is not defined

The issue is that the `docx` UMD library loads as `window.docx` but the script runs before it's fully available, or the CDN variable name differs. Let me fix this by checking the library loads correctly and accessing it properly.

Check current file >

The problem is that `docx` version 8.5.0 may not be available on cdnjs, or the UMD global is named differently. Let me switch to a reliable version and use `window.docx` explicitly, with a fallback check.

Edited 2 files, read a file >

Two changes made:

1. **Switched the `docx` CDN** from cdnjs (where version 8.5.0 wasn't available) to `unpkg.com`, which reliably hosts it.
2. **Accessed it as `window.docx`** explicitly, with a clear error message if it still fails to load.

If you still see an error, it would mean the unpkg CDN is being blocked — let me know and I can bundle the logic differently.

Excellent—version #1 does exactly as asked.
Can you make a version #2 that does the following?
When the user downloads the docx, two docx files should be downloaded:
(1) an answer_key with the correct answers still marked with [Answer.]
(2) a test_form with the [Answer.] markers removed.

Viewed a file, created a file, read a file >

Here's MCQer v2. The key design decision: both files are generated from the **same scrambled order**, so the answer key and test form are in sync — the position of the correct answer is identical in both, just marked in one and not the other.

- `answer_key.docx` — options scrambled, correct answer prefixed with `[Answer.]`
- `test_form.docx` — same scrambled order, `[Answer.]` marker absent

Both downloads trigger automatically with a brief stagger to prevent browsers from blocking the second one.

Excellent—version #2 does exactly as asked, and the design decision is correct.
Can you make a version #3 that adds the following:
The user selects a number of exam versions to download, from 2 to 5. The default number should be 3.
When the user downloads the documents, each version comes as an answer_key and test_form. The difference between the versions is that the order of the answer-options is scrambled differently. Each version's key should still match its test_form.
The versions should be labelled as versionA, versionB, versionC, and so forth.

Created a file, read a file >

Here's MCQer v3. A few notes on the implementation:

- The **version stepper** (− / +) lets you pick 2–5 versions, defaulting to 3. The badges update live to show which version letters will be generated.
- Each version's scramble is **independently randomized**, so versionA, versionB, etc. will each have a different option order.
- Files download **sequentially** (one at a time with a short stagger) to prevent browsers from blocking multiple simultaneous downloads. With 5 versions that's 10 files, so there may be a brief pause.
- Files are named `versionA_answer_key.docx`, `versionA_test_form.docx`, `versionB_answer_key.docx`, and so on.

Excellent—version #3 does exactly as asked.
Can you make a version #4 that adds the following features?
Answer-options should be numbered by letter (a) (b) (c) (d) or (e), formatted as follows.
[Answer.] (a) Energy production.
[Distractor.] (b) Protein synthesis.
Scrambling should not confuse the order of options, and should preserve as follows:
[Distractor.] (a) Protein synthesis.
[Answer.] (b) Energy production.
The MCQer should be able to handle the case where a question is uploaded with a [Question.] but no [Distractor.] These questions should still be found. If no distractors are present, the question is not a multiple-choice question and requires no scrambling, but should still appear in all test versions.
All test_form and answer_key should be downloaded in both docx and PDF formats.
The downloaded documents should have their coversheets appended as described below:
The user uploads an optional coversheet in docx format.
If a coversheet is uploaded:
The coversheet should be concatenated to the front of all answer_keys and test_forms.
The coversheet should be adjusted so that the text [Version X] is replaced with the version letter. So, for version A, [Version X] would be replaced with the text Version A.

Show less

Read the docx skill for best practices >

Good. This is a complex client-side HTML file. Let me plan the approach carefully:

1. **Letter numbering**: Options get `(a)`, `(b)`, etc. assigned *after* scrambling, so the letter always reflects position, not identity.
2. **No-distractor questions**: Parse and include them as-is (just a question, no options), no scrambling needed.
3. **PDF output**: Client-side PDF from docx content using `jsPDF` — I'll render the content directly since we can't run LibreOffice in the browser.
4. **Coversheet**: Read uploaded docx XML with JSZip, do find-replace of `[Version X]` → `Version A` etc., then prepend those paragraphs to the main content sections.

The PDF generation from docx in-browser is the trickiest part. I'll use `jsPDF` to render the same content as the docx (same questions/answers), which is the most reliable client-side approach.

Created a file, read a file >

Here's MCQer v4. A few notes on the implementation decisions:

Letter numbering: Letters `(a)`, `(b)`, etc. are assigned *after* scrambling, so the letter always reflects the option's position in that version. The answer key marks the correct answer with `[Answer.]` before the letter, e.g. `[Answer.] (b) Energy production.`

No-distractor questions: Any paragraph with `[Question.]` but no `[Answer.]` or `[Distractor.]` tags is still found and included in all versions, appearing as a plain Heading 1 with no options beneath it. The preview flags these as "No options — will appear unscrambled."

PDF generation: PDFs are rendered directly in the browser using jsPDF — questions in bold 14pt, options indented in 12pt. Since jsPDF can't read docx styles, the coversheet text is rendered as plain 12pt paragraphs followed by a page break.

Coversheet substitution: `[Version X]` anywhere in the coversheet is replaced with `Version A`, `Version B`, etc. The coversheet text is prepended to both the docx and PDF, separated by a page break from the questions.

Downloads: Each version produces 4 files (`answer_key.docx`, `answer_key.pdf`, `test_form.docx`, `test_form.pdf`), downloaded sequentially with a progress bar.

This is almost perfect.
I have some questions about the cover sheet before requesting a version #5.
Does a way exist to preserve the styles of the cover sheet?
Alternatively, does a way exist to upload a PDF coversheet, for which [Version X] will still be replaced with the version letter?

Great questions. Here's an honest assessment of both options:

Preserving styles from a docx coversheet is genuinely difficult client-side. The docx format stores styles in XML, and while you could unzip the docx with JSZip and extract the raw paragraph XML, splicing foreign XML from one docx into another is fragile — relationship IDs, style definitions, font tables, and numbering configs all need to be reconciled between the two files. It's doable in principle but prone to corruption, and the result in Word can be unpredictable. I wouldn't recommend it for a tool meant to produce reliable outputs.

Replacing text in a PDF coversheet is also problematic. PDFs don't store text as editable strings the way docx does — text is encoded as positioned drawing instructions, often with fonts embedded as subsets. Finding and replacing `[Version X]` in raw PDF bytes is unreliable and can corrupt the file. There's no good client-side JS library for this.

What I'd actually recommend for v5 is one of these two cleaner approaches:

The first option is to keep the coversheet as a docx but render it into the PDF using a more faithful conversion — specifically, reading the docx XML with JSZip and mapping the paragraph styles (Heading 1, Heading 2, Normal, bold runs, etc.) to jsPDF calls. This preserves the visual structure of the coversheet reasonably well in both the docx output (where it's already good) and the PDF output.

The second option, and probably the most robust, is to have the user supply the coversheet as an **HTML file** instead. HTML coversheets are easy to style richly, the `[Version X]` replacement is trivial string substitution, and jsPDF has a `html()` method that renders styled HTML directly to PDF with good fidelity. The same HTML content can also be converted to docx paragraph objects with reasonable style mapping.

Which of these appeals to you, or would you like a different approach?

Does a way exist to add a PDF coversheet to the PDF answer_keys and test_forms only (and not to the docx output files)—concatenating it to them without reading the PDF in detail?

Yes — this is actually the cleanest approach of all. PDF concatenation (merging two PDFs by appending one's pages to another's) doesn't require reading or understanding the content at all. You just combine the page streams. The library **pdf-lib** handles this entirely client-side and is well-suited to it:

```javascript
import { PDFDocument } from 'pdf-lib';

const cover = await PDFDocument.load(coversheetBytes);
const body  = await PDFDocument.load(bodyBytes);

const merged = await PDFDocument.create();
const coverPages = await merged.copyPagesFrom(cover, cover.getPageIndices());
const bodyPages  = await merged.copyPagesFrom(body,  body.getPageIndices());
coverPages.forEach(p => merged.addPage(p));
bodyPages.forEach(p  => merged.addPage(p));

const mergedBytes = await merged.save();
```

The `[Version X]` replacement is still not possible this way — the PDF is treated as opaque and its contents aren't touched. But from what you've described, the coversheet would be the same for all versions except for the version letter, so you'd need either one pre-made PDF per version, or to leave the version label blank on the coversheet and let the test-taker fill it in.

If you can live with that constraint, v5 would work like this: the user uploads a PDF coversheet, it gets concatenated as-is to the front of each PDF output, and the docx outputs get no coversheet at all. Would you like to proceed on that basis?

Yes.
Can you generate a version #5 as follows?
The version letter will appear centered at the top of the list of questions.
If the user includes it, it will appear in the uploaded question-list as [Version X].
The version letter should appear in both answer_key and test_form
If the uploaded docx does not include [Version X], MCQer should add a version letter anyway to the output documents at the top of the questions.
MCQer should take an optional PDF coversheet upload.
In addition, can you adjust the following?
A question should never appear on a different page from its corresponding [Answer.] and [Distractor.]
The tag [Page break.] should cause a page break. This tag will appear mainly at the end of no-distractor questions (which will usually be long-answer questions that the students write below).

Show less

Review v4 source to build v5 from it >

Good. Now let me plan the key changes for v5 carefully:

1. **Version label at top of questions** — centered Heading 1 paragraph saying "Version A" etc., replacing `[Version X]` if present in source, or prepending if not.
2. **PDF coversheet concatenation** — use `pdf-lib` (via CDN) to merge the uploaded PDF coversheet pages before the generated body PDF. No docx coversheet at all.
3. **Keep questions with their options on the same page** — in jsPDF, pre-calculate the height of the full question block before drawing; if it doesn't fit, add a page first. In docx, use `keepNext: true` on the question paragraph and `keepLines: true` on the option group.
4. `[Page break.]` **tag** — recognized as a special paragraph type producing a page break in both docx and PDF.
5. **Remove the docx coversheet logic** from v4 entirely.

Created a file, read a file >

Here's MCQer v5. Notes on the key implementation decisions:

Version label: `buildVersionParas()` always inserts a centered `Version A` / `Version B` etc. as the first paragraph. When the source contains `[Version X]` as a standalone paragraph, it's detected and stripped from the question list (so it doesn't get parsed as a stray line), and the label is injected in its place at the top. When it's absent, the label is simply prepended. Either way, both the answer key and test form get it.

Keep question with options: In the docx, `keepNext: true` is set on the question paragraph and on every option paragraph except the last, forming an unbreakable chain. In the PDF, before drawing a question block, the renderer pre-calculates the total height of the question plus all its options and forces a page break first if the block won't fit.

`[Page break.]`: Recognized as a standalone paragraph tag, producing a `PageBreak` in docx and a `doc.addPage()` in the PDF renderer. It works anywhere in the question list, including after no-distractor questions.

PDF coversheet: Uploaded as a PDF, stored as raw bytes, and concatenated using `pdf-lib`'s `copyPagesFrom` — the coversheet content is never read or modified. No coversheet is attached to the docx outputs.

I get an error: Error: merged.copyPagesFrom is not a function

The issue is that `pdf-lib` loads its exports under `window.PDFLib` but `PDFDocument` needs to be accessed correctly, and the method is `copyPages` not `copyPagesFrom`. Let me fix the merge function.

Viewed 3 files >

The correct pdf-lib API is `merged.copyPages(source, indices)` — not `copyPagesFrom`. Let me fix both calls:

Edited a file, read a file >

Fixed — the pdf-lib API is `copyPages(sourceDoc, indicesArray)` on the destination document, not `copyPagesFrom`. Both the coversheet and body merge calls are now corrected.

PDF concatenation successful. Can you make the following adjustments for version #6?
[Page break.] should be processed as a page break even if it does not occur as a separate paragraph.
MCQer should number questions in consecutive order. Each question should be given a number, resulting in question formatting like the following:
1. What is the primary function of the mitochondria in a cell?

Viewed 2 files >

Good. I have a clear picture. The two changes are:

1. `[Page break.]` **inline**: Instead of only matching it as a standalone paragraph, split every paragraph on `[Page break.]` before parsing, emitting a pagebreak item between the parts.
2. **Question numbering**: Track a counter in `buildVersionParas` and prepend `1.`, `2.`, etc. to each question's text.

Edited 2 files, ran a command >

Now update the call site — `parseParagraph` now returns an array, so we need to flatten:

Update parseParagraph call site to flatten arrays >

Now add question numbering in `buildVersionParas`:

Add consecutive question numbering in buildVersionParas >

Now update the title and subtitle to say v6:

Edited 2 files >

Now copy to outputs and verify the key changes look correct:

Copy to outputs and verify key changes >

All three changes are confirmed in place.

Presented file >

Here's MCQer v6. Two targeted changes from v5:

Inline `[Page break.]`: The parser now splits every paragraph on `[Page break.]` before doing anything else, using `flatMap` to flatten the resulting arrays. So `[Question.] Long answer question here. [Page break.]` correctly emits a question followed by a page break, and `[Page break.]` alone still works as before.

Question numbering: A `questionNumber` counter increments for every question item (page breaks don't affect it), and the number is prepended as `1.`, `2.`, etc. when building each version's paragraph list. Since numbering happens at build time rather than parse time, the numbers are consistent across all versions regardless of scrambling.

Version #6 is excellent.
For a version #7, can you adjust the colour scheme?
You have the following colours to work with, as reported in this changelog from a separate but related design project:
"Background flipped to a warm off-white (`#F7F5F0`), with white card surfaces — fully light-mode
All primary text is now Navy `#0C2F56` — crisp and dark
Student names in the grid use Navy on a light blue-tinted background (`#EEF3F8`) — no more faded text, fully readable at a glance
Gold is used for upload card confirmation status and accent borders (darkened slightly from `#F2CC56` to `#D4A800` for contrast on white)
Red `#B2241A` is used for the eyebrow label, the in the heading, the PDF button, drop-target highlights, and the swap count badge
Empty seat cells are a very subtle grey — present but clearly not filled"
The thematic colours for my institution are:
`#0C2F56`
Dark Red
`#B2241A`
Gold/Yellow
`#F2CC56`
White
`#FFFFFF`
Deep Blue (Waves)
`#152A4E`
Can you also convey the expected formatting of questions in the question sheet?

Show less

Review the CSS and HTML structure of v6 to plan the reskin >

Good. Now I have a clear picture. Let me plan the reskin carefully:

- **Background**: warm off-white `#F7F5F0`
- **Card**: white `#FFFFFF`
- **All primary text**: Navy `#0C2F56`
- **Upload label eyebrow text**: Dark Red `#B2241A`
- **Drop zone default border**: light blue-tinted `#C8D8E8`
- **Drop zone hover/dragover**: Red `#B2241A` border + light red tint
- **Drop zone confirmed**: Gold border `#D4A800` + warm gold-tinted bg
- **CTA link text**: Navy `#0C2F56`
- **Badges**: Navy bg `#0C2F56` white text (or gold accent)
- **Version row**: light blue-tinted bg `#EEF3F8`
- **Generate button**: Navy `#0C2F56`, hover Deep Blue `#152A4E`
- **Progress bar**: Gold `#F2CC56`
- **Status info**: blue-tinted; success: gold-tinted; error: red-tinted
- **Preview list border/items**: light blue tint
- **Format preview**: show a mini mock-up of how a question looks in the output

Also adding a format preview section showing the expected question layout.

Ran a command, edited a file >

Now update the HTML body — masthead, format reference panel, and version label:

Update masthead and add format reference panel to HTML body >

Now show the format reference panel when questions are loaded:

Show format-ref panel when questions load >

Also hide format-ref and other panels on reset:

Edited a file, ran a command, read a file >

Here's MCQer v7. Summary of changes:

Colour scheme — fully remapped to your institutional palette:

- Warm off-white `#F7F5F0` background, white card
- Navy `#0C2F56` for all primary text, buttons, and version badges (with gold `#F2CC56` text on the badges)
- Dark Red `#B2241A` for the eyebrow label, section headers, drag highlight, and error states
- Gold `#D4A800` / `#F2CC56` for the confirmed-upload border, progress bar, and success states
- Deep Blue `#152A4E` for button hover

Format reference panel — appears after a question file is loaded, showing a compact annotated mock-up of exactly how the output documents will look: version label centred at top, numbered MCQ with lettered options (answer key version shows `[Answer.]` prefix in gold), a long-answer question with a note, and a page-break marker. This gives the user a clear reference without needing to generate a file first.

Version #7 is excellent.
Can you made a version #8 with the following features added?
A question prepended with [Only Version X.] should appear only in that version.
For example:
[Only Version X.] [Question.] What gas do plants absorb during photosynthesis? [Answer.] Carbon dioxide. [Distractor.] Oxygen. [Distractor.] Nitrogen. [Distractor.] Hydrogen.
For a series of questions each marked [Option.] and following the separate-paragraph tag [Each version take Y_number of the following options.], include Y_number of the option questions in that version.
The option questions should be distributed as evenly as possible among the versions, and each version should be given exactly the same number of the option questions. Page breaks should be respected per-version and question numbering should still make sense for each test_form and its corresponding answer_key.
For example, in the following, each version should have 2 of the 3, with all of the 3 used among the various versions.
[Each version take 2 of the following options.]
[Option.] [Question.] What gas do plants absorb during photosynthesis? [Answer.] Carbon dioxide. [Distractor.] Oxygen. [Distractor.] Nitrogen. [Distractor.] Hydrogen.
[Option.] [Question.] What is the primary function of the mitochondria in a cell? [Answer.] Energy production. [Distractor.] Protein synthesis. [Distractor.] DNA replication. [Distractor.] Cell division.
[Option.] [Question.] What gas is CO2? [Answer.] Carbon dioxide. [Distractor.] Oxygen. [Distractor.] Nitrogen. [Distractor.] Hydrogen.

Show less

This is a well-defined problem. Let me plan the logic carefully before coding.

Parsing additions:

- `[Only Version X.]` prefix → question gets a `onlyVersion: 'X'` property, filtered per version
- `[Each version take N of the following options.]` → a standalone paragraph that opens an "option pool"
- `[Option.]` prefix → question belongs to the current open option pool
- The pool closes when a non-`[Option.]` paragraph is encountered

Option pool distribution logic:

- Given a pool of M questions and N versions each taking K, distribute as evenly as possible
- Total slots = N×K; if M < N×K, warn; if M > N×K, some questions go unused (by design)
- Strategy: assign questions round-robin across versions using a shuffled pool order, so each question appears in as many versions as needed (K×N/M times, roughly), and each version gets exactly K

Numbering: happens at build time per version after filtering, so it's always consecutive and correct.

Viewed 2 files, ran a command >

Now I have everything I need. The key design decisions:

- `parsedQuestions` will contain a new item type `{ type: 'option-pool', take: N, questions: [...] }`
- `[Only Version X.]` adds `onlyVersion: 'X'` to the question
- `buildVersionParas` is called with a pre-computed assignment of which option pool questions each version gets
- The assignment is computed once before all versions are built, using a round-robin shuffle across versions

Replace parseParagraph and the questions file handler with v8 logic supporting [Only Version X.], [... >

Now update `renderPreview` to show option pool info, and update `buildVersionParas` to use pool assignments and filter `onlyVersion`:

Update renderPreview for new item types >

Now update `buildVersionParas` to handle pool assignments and `onlyVersion` filtering:

Replace buildVersionParas with v8 version that handles pools and onlyVersion >

Now update the main generate handler to compute pool assignments once and pass them in:

Viewed 2 files, edited a file >

Now update the title/version label and update the format-ref panel to show the new tags:

Edited 3 files, ran 2 commands >

All wired correctly.

Presented file >

Here's MCQer v8. Notes on the new features:

`[Only Version X.]` — prefix any paragraph with this tag (where X is A–E) and it will be silently omitted from all other versions. Question numbering re-counts per version, so gaps never appear.

Option pools — `[Each version take N of the following options.]` opens a pool; every immediately following `[Option.]` paragraph is collected into it. The pool closes as soon as a non-`[Option.]` paragraph is encountered. Pool assignment happens once before any version is built, using a round-robin shuffle across versions so questions are distributed as evenly as possible. For your 3-question, take-2 example: versions A/B/C would each get a different pair, with every question appearing in exactly 2 of the 3 versions.

Numbering — because filtering and pool selection happen per-version before numbering, every test form and its corresponding answer key always have consecutive numbers with no gaps, regardless of which questions were included or excluded.

The preview panel now shows option pools visually grouped with a red label showing how many each version will take, and version-specific questions are tagged in red.